

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0503

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	5

8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	5
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	5
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

Code of Conduct:

*I _B.Poorvaja (192473008)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. RELATIONAL ALGEBRA OPERATIONS :

Employee(EmpID, EmpName, Salary, DeptID)

Department(DeptID, DeptName)

a) Select (σ)

Employees with salary greater than 50000:

$$\sigma_{\text{Salary} > 50000}(\text{Employee})$$

b) Project (π)

Display employee names:

$$\pi_{\text{EmpName}}(\text{Employee})$$

c) Union ($\cup$)

Employees in HR or IT departments:

$$\pi_{\text{EmpID}}(\sigma_{\text{DeptID}='HR'}(\text{Employee})) \cup \pi_{\text{EmpID}}(\sigma_{\text{DeptID}='IT'}(\text{Employee}))$$

d) Difference (-)

Employees not working in IT:

$$\pi_{\text{EmpID}}(\text{Employee}) - \pi_{\text{EmpID}}(\sigma_{\text{DeptID}='IT'}(\text{Employee}))$$

e) Cartesian Product ($\times$)

$$\text{Employee} \times \text{Department}$$

f) Join (⋈)

Employee ⋈_{Employee.DeptID=Department.DeptID} Department

2. SQL QUERY:

```
CREATE TABLE DEPARTMENT (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50) NOT NULL UNIQUE  
);
```

```
CREATE TABLE STUDENT (  
    SID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    DOB DATE,  
    DeptID INT,  
    FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)  
);
```

```
mysql> CREATE TABLE STUDENT (  
->     SID INT PRIMARY KEY,  
->     Name VARCHAR(50) NOT NULL,  
->     DOB DATE,  
->     DeptID INT,  
->     FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> DESC STUDENT;  
+-----+-----+-----+-----+-----+-----+  
| Field | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| SID   | int           | NO   | PRI | NULL    |       |  
| Name  | varchar(50)   | NO   |     | NULL    |       |  
| DOB   | date          | YES  |     | NULL    |       |  
| DeptID | int           | YES  | MUL | NULL    |       |  
+-----+-----+-----+-----+-----+-----+  
4 rows in set (0.00 sec)
```

```
mysql> DESC DEPARTMENT;
```

Field	Type	Null	Key	Default	Extra
DeptID	int	NO	PRI	NULL	
DeptName	varchar(50)	NO		NULL	

```
2 rows in set (0.02 sec)
```

3. SQL QUERIES :

COUNT() :

```
SELECT SalesPerson, COUNT(*) AS TotalSales
FROM SALES
GROUP BY SalesPerson;
```

```
mysql> SELECT SalesPerson, COUNT(*) AS TotalSales
-> FROM SALES
-> GROUP BY SalesPerson;
```

SalesPerson	TotalSales
Amit	2
Rahul	2
Sneha	1

```
3 rows in set (0.01 sec)
```

SUM() :

```
SELECT Product, SUM(Amount)
FROM SALES
GROUP BY Product;
```

```
mysql> SELECT Product, SUM(Amount)
-> FROM SALES
-> GROUP BY Product;
```

Product	SUM(Amount)
Laptop	150000.00
Mouse	2500.00
Chair	8000.00
Table	10000.00

```
4 rows in set (0.00 sec)
```

AVG() :

```
SELECT Category, AVG(Amount)
FROM SALES
GROUP BY Category;
```

```
mysql> SELECT Category, AVG(Amount)
-> FROM SALES
-> GROUP BY Category;
+-----+-----+
| Category | AVG(Amount) |
+-----+-----+
| Electronics | 50833.333333 |
| Furniture | 9000.000000 |
+-----+-----+
2 rows in set (0.01 sec)
```

MAX() :

```
SELECT Product, MAX(Amount)
FROM SALES
GROUP BY Product;
```

```
mysql> SELECT Product, MAX(Amount)
-> FROM SALES
-> GROUP BY Product;
+-----+-----+
| Product | MAX(Amount) |
+-----+-----+
| Laptop | 100000.00 |
| Mouse | 2500.00 |
| Chair | 8000.00 |
| Table | 10000.00 |
+-----+-----+
4 rows in set (0.01 sec)
```

MIN() :

```
SELECT Product, MIN(Amount)
FROM SALES
GROUP BY Product;
```

```
mysql> SELECT Product, MIN(Amount)
-> FROM SALES
-> GROUP BY Product;
+-----+-----+
| Product | MIN(Amount) |
+-----+-----+
| Laptop | 50000.00 |
| Mouse | 2500.00 |
| Chair | 8000.00 |
| Table | 10000.00 |
+-----+-----+
4 rows in set (0.00 sec)
```

HAVING :

```
SELECT Product, SUM(Amount)
FROM SALES
GROUP BY Product
HAVING SUM(Amount) > 5000;
```

```
mysql> SELECT Product, SUM(Amount)
-> FROM SALES
-> GROUP BY Product
-> HAVING SUM(Amount) > 5000;
+-----+-----+
| Product | SUM(Amount) |
+-----+-----+
| Laptop | 150000.00 |
| Chair | 8000.00 |
| Table | 10000.00 |
+-----+-----+
3 rows in set (0.01 sec)
```

```
mysql> SELECT Product,
-> COUNT(*) AS TotalOrders,
-> SUM(Amount) AS TotalSales,
-> AVG(Amount) AS AverageSales
-> FROM Sales
-> GROUP BY Product
-> HAVING SUM(Amount) > 10000;
```

Product	TotalOrders	TotalSales	AverageSales
Laptop	2	150000.00	75000.000000

4. QUERY:

```
SELECT EmpID,
       EmpName,
       Salary,
       DeptID
FROM Employee E
WHERE Salary >
(
SELECT AVG(Salary)
FROM Employee
WHERE DeptID = E.DeptID
);
```

```
mysql> SELECT EmpID,
-> EmpName,
-> Salary,
-> DeptID
-> FROM Employee E
-> WHERE Salary >
-> (
-> SELECT AVG(Salary)
-> FROM Employee
-> WHERE DeptID = E.DeptID
-> );
```

EmpID	EmpName	Salary	DeptID
1	Arun	55000	1
4	Sneha	70000	2

2 rows in set (0.01 sec)

5. TYPES OF JOIN OPERATIONS:

INNER JOIN:

```
SELECT *
FROM Employee E
INNER JOIN Project P
ON E.EmpID=P.EmpID;
```

EmpID	EmpName	Salary	DeptID	ProjectID	ProjectName	EmpID
1	Arun	55000	1	101	Library System	1
2	Divya	65000	2	102	Hospital System	2
4	Sneha	70000	2	103	Banking App	4

LEFT OUTER JOIN:

```
SELECT *
FROM Employee E
LEFT JOIN Project P
ON E.EmpID=P.EmpID;
```

EmpID	EmpName	Salary	DeptID	ProjectID	ProjectName	EmpID
1	Arun	55000	1	101	Library System	1
2	Divya	65000	2	102	Hospital System	2
3	Kumar	45000	1	NULL	NULL	NULL
4	Sneha	70000	2	103	Banking App	4
5	Vijay	50000	3	NULL	NULL	NULL

RIGHT OUTER JOIN :

```
SELECT *
FROM Employee E
RIGHT JOIN Project P
ON E.EmpID=P.EmpID;
```

EmpID	EmpName	Salary	DeptID	ProjectID	ProjectName	EmpID
1	Arun	55000	1	101	Library System	1
2	Divya	65000	2	102	Hospital System	2
4	Sneha	70000	2	103	Banking App	4

FULL OUTER JOIN :

```
SELECT *
FROM Employee E
FULL OUTER JOIN Project P
ON E.EmpID=P.EmpID;
```

EmpID	EmpName	Salary	DeptID	ProjectID	ProjectName	EmpID
1	Arun	55000	1	101	Library System	1
2	Divya	65000	2	102	Hospital System	2
3	Kumar	45000	1	NULL	NULL	NULL
4	Sneha	70000	2	103	Banking App	4
5	Vijay	50000	3	NULL	NULL	NULL

5 rows in set (0.01 sec)

CROSS JOIN :

```
SELECT *  
FROM Employee  
CROSS JOIN Project;
```

EmpID	EmpName	Salary	DeptID	ProjectID	ProjectName	EmpID
1	Arun	55000	1	103	Banking App	4
1	Arun	55000	1	102	Hospital System	2
1	Arun	55000	1	101	Library System	1
2	Divya	65000	2	103	Banking App	4
2	Divya	65000	2	102	Hospital System	2
2	Divya	65000	2	101	Library System	1
3	Kumar	45000	1	103	Banking App	4
3	Kumar	45000	1	102	Hospital System	2
3	Kumar	45000	1	101	Library System	1
4	Sneha	70000	2	103	Banking App	4
4	Sneha	70000	2	102	Hospital System	2
4	Sneha	70000	2	101	Library System	1
5	Vijay	50000	3	103	Banking App	4
5	Vijay	50000	3	102	Hospital System	2
5	Vijay	50000	3	101	Library System	1

6. VIEW CREATION:

```
CREATE VIEW DeptSalaryView AS  
SELECT D.DeptName,  
COUNT(E.EmpID) AS TotalEmployees,  
AVG(E.Salary) AS AverageSalary  
FROM Employee E  
JOIN Department D  
ON E.DeptID=D.DeptID  
GROUP BY D.DeptName;
```

QUERY ON VIEW:

```
SELECT *  
FROM DeptSalaryView  
WHERE AverageSalary>50000;
```

DeptName	TotalEmployees	AverageSalary
IT	2	67500.0000

7. RELATIONAL ALGEBRA EXPRESSION:

Relations:

Student(SID, Name)

Enrollment(SID, CID)

Course(CID)

Students enrolled in all courses:

$$\pi_{Name} \left(Student \bowtie \left(\pi_{SID,CID}(Enrollment) \div \pi_{CID}(Course) \right) \right)$$

8. SQL DML STATEMENTS :

INSERT :

INSERT INTO Student

VALUES

(105,'Riya','2005-07-20',2);

```
mysql> SELECT * FROM Student;
+-----+-----+-----+-----+
| SID | Name | DOB       | DeptID |
+-----+-----+-----+-----+
| 105 | Riya | 2005-07-20 | 2      |
+-----+-----+-----+-----+
```

UPDATE :

UPDATE Student

SET DeptID = 3

WHERE SID = 105;

```
mysql> SELECT * FROM Student;
+-----+-----+-----+-----+
| SID | Name | DOB       | DeptID |
+-----+-----+-----+-----+
| 105 | Riya | 2005-07-20 | 3      |
+-----+-----+-----+-----+
```

DELETE :

DELETE FROM Student

WHERE SID = 105;

```
mysql> DELETE FROM Student
-> WHERE SID = 105;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Student;
Empty set (0.00 sec)
```

9. TUPLE RELATIONAL CALCULUS:

$$\{ t \mid Employee(t) \wedge t.Dept = 'IT' \wedge t.Salary > 50000 \}$$

(Or)

$$\{ t \mid Employee(t) \wedge t.Dept = 'IT' \wedge t.Salary > 50000 \}$$

Tuple Relational Calculus Expression :

```
{ E.EmpName |  
  EMPLOYEE(E) ∧  
  ∃D (DEPARTMENT(D) ∧  
    E.DeptID = D.DeptID ∧  
    D.DeptName = 'IT' ∧  
    E.Salary > 50000)  
}
```

10. AIRLINE RESERVATION SYSTEM

Relational Schema :

Passenger

```
Passenger(  
  PassengerID PRIMARY KEY,  
  PassengerName,  
  Phone,  
  Email  
)
```

Flight

```
Flight(  
  FlightID PRIMARY KEY,  
  FlightNo,  
  Source,  
  Destination,  
  DepartureTime,  
  ArrivalTime  
)
```

Booking

```
Booking(  
BookingID PRIMARY KEY,  
PassengerID,  
FlightID,  
BookingDate,  
SeatNo,  
Status,  
FOREIGN KEY(PassengerID)  
REFERENCES Passenger(PassengerID),  
FOREIGN KEY(FlightID)  
REFERENCES Flight(FlightID)  
)
```

SQL Query 1 (Join)

```
SELECT PassengerName,  
FlightNo,  
Source,  
Destination  
FROM Passenger P  
JOIN Booking B  
ON P.PassengerID=B.PassengerID  
JOIN Flight F  
ON B.FlightID=F.FlightID;
```

PassengerName	FlightNo	Source	Destination
Rahul	AI101	Chennai	Delhi
Anu	AI101	Chennai	Delhi
Priya	AI102	Chennai	Mumbai

Query 2 (Subquery):

```
SELECT FlightNo  
FROM Flight  
WHERE FlightID IN  
(  
SELECT FlightID  
FROM Booking  
GROUP BY FlightID  
HAVING COUNT(*) > 1)
```

);

```
1 row in set (0.00
```

FlightNo
AI101

Query 3 (Aggregate) :

```
SELECT FlightID,  
COUNT(*) AS TotalBookings  
FROM Booking  
GROUP BY FlightID;
```

```
-> GROUP BY FlightID;
```

FlightID	TotalBookings
101	2
102	1

Query 4 (View) :

```
CREATE VIEW BookingDetails AS  
SELECT PassengerName,  
FlightNo,  
SeatNo,  
Status  
FROM Passenger P  
JOIN Booking B  
ON P.PassengerID = B.PassengerID  
JOIN Flight F  
ON B.FlightID = F.FlightID;
```

Query 5 (Query on the View) :

```
SELECT *  
FROM BookingDetails  
WHERE Status='Confirmed';
```

```
-> WHERE Status='Confirmed';
```

PassengerName	FlightNo	SeatNo	Status
Rahul	AI101	A1	Confirmed
Priya	AI102	B2	Confirmed

2 rows in set (0.01 sec)